

The `cls` parameter acts like `self` except `self` refers to an object, but the `cls` parameter refers to an object's class. This means that the code in a class method cannot access an individual object's attributes or call an object's regular methods. Class methods can only call other class methods or access class attributes. We use the name `cls` because `class` is a Python keyword, and just like other keywords, such as `if`, `while`, or `import`, we can't use it for parameter names. We often call class attributes through the class object, as in `ExampleClass.exampleClassMethod()`. But we can also call them through any object of the class, as in `obj.exampleClassMethod()`.

Class methods aren't commonly used. The most frequent use case is to provide alternative constructor methods besides `__init__()`. For example, what if a constructor function could accept either a string of data the new object needs or a string of a filename that contains the data the new object needs? We don't want the list of the `__init__()` method's parameters to be lengthy and confusing. Instead let's use class methods to return a new object.

For example, let's create an `AsciiArt` class. As you saw in Chapter 14, ASCII art uses text characters to form an image.

```
class AsciiArt:
    def __init__(self, characters):
        self._characters = characters

    @classmethod
    def fromFile(cls, filename):
        with open(filename) as fileObj:
            characters = fileObj.read()
            return cls(characters)

    def display(self):
        print(self._characters)

    # Other AsciiArt methods would go here...

face1 = AsciiArt(' _____\n' +
                 '|  . .  |\n' +
                 '|  \_\_/  |\n' +
                 '|_____|')
face1.display()
```